

CogPath: A Constraint-Guided Context-Reduction Framework for LLM-Based Test Generation

Anonymous Authors

Abstract—Large language models (LLMs) have shown promise in automated unit test generation, yet achieving high branch coverage for complex methods remains challenging. We attribute this limitation not to model capability alone, but to a fundamental cognitive misalignment between what LLM prompts expose and what branch reachability requires: implicit cross-method preconditions are hidden, while branch-irrelevant code is over-provided. To address this misalignment, we propose COGPATH, a unit test generation framework that externalizes hidden path conditions as Constraint-Hints and isolates branch-relevant dependencies via coverage-driven Backward Slicing, within an iterative generate-execute-repair loop. Our empirical evaluation, conducted on 2971 methods with cyclomatic complexity greater than 10 from 14 open-source projects, demonstrates that COGPATH achieves 12.8% higher line coverage and 10.2% higher branch coverage compared to the state-of-the-art, with improvements reaching 32.3% and 20% respectively on the most complex project.

Index Terms—Unit Test Generation, Large Language Models, Software Testing, Program Slicing

I. INTRODUCTION

Automated unit test generation plays a critical role in improving software reliability and reducing developer effort. Traditional techniques, such as symbolic execution and search-based software testing (SBST), have made substantial progress in systematically exploring input spaces and maximizing structural coverage [1]–[4]. However, these approaches often face fundamental limitations in scalability and in handling complex program semantics: They struggle to capture implicit preconditions that arise from intricate object states and inter-procedural interactions, where the conditions required to reach a target path are established outside the local method scope through sequences of prior method calls [5].

Recent advances demonstrate that large language models (LLMs) can directly synthesize executable unit tests, significantly alleviating manual effort [6]–[8]. To handle complex methods and improve coverage, subsequent research mainly pursues two directions. The first is *feedback-driven refinement*: CoverUp [7] iteratively targets untested regions using dynamic coverage analysis, while YATE [9] repairs failing tests to enhance validity. The second direction emphasizes *structural decomposition and path sensitivity*. HITS [10] prompts LLMs to decompose methods into fragments. Advancing this using program analysis, SymPrompt [11] extracts branch constraints from Control Flow Graphs (CFGs), whereas Panta [12] fuses static CFG analysis with dynamic coverage feedback to identify uncovered paths. Building upon this trajectory, JUnit-Genie [13] achieves strict path sensitivity by isolating the minimal, path-specific context required for targeted generation.

Despite these promising results, a substantial coverage gap remains, rooted not in a lack of feedback or prompting sophistication, but in a deeper mismatch, i.e., *cognitive misalignment*, between the information exposed to the LLM and what is actually required to cover complex target branches.

① One important dimension of this misalignment is *What to Satisfy*. In many cases, branch reachability depends on implicit preconditions scattered across long dependency chains [14], requiring the model to infer hidden constraints through multiple variable assignments and state transitions before it can synthesize a valid test. Existing approaches typically provide only the target branch structure and compress constraint inference and test synthesis into a single generation step, without any explicit intermediate output or independent verification of the inferred constraints. As a result, the LLM can easily skip or compress critical reasoning steps, producing tests that appear plausible but fail to satisfy the actual conditions required to reach the target branch. This problem becomes particularly severe when the underlying dependency chain is long or semantically complex.

② The second dimension concerns *What to Attend To*. Existing methods usually expose the entire method body to the LLM, even though only a small subset of statements has real data or control dependency on the target branch. The resulting context is therefore structurally misaligned with the branch-coverage objective: coverage-critical dependencies are diluted by irrelevant code, attention is spread across non-essential statements, and the model must reason under a representation that over-emphasizes what is visible rather than what is truly necessary [15]. This not only amplifies cognitive burden, but also makes precise state construction substantially less likely. Even without explicit distractors, LLM performance can already degrade as context length grows [16], [17]; in branch-oriented test generation, this effect is further magnified because the relevant context is often deeply buried inside much longer method bodies.

Approach. To address this cognitive misalignment, we propose COGPATH, a branch-level alignment framework for LLM-based unit test generation. Rather than treating an uncovered branch as a generic request for more tests, COGPATH aligns the prompt with the specific reasoning needed to reach that branch: it exposes the preconditions that must be satisfied and suppresses context that is irrelevant to the branch outcome. This alignment is performed inside an iterative generate-execute-repair loop driven by dynamic coverage feedback. For each resistant branch, COGPATH derives path-specific *Constraint-Hints* that translate hidden reachability

requirements—such as setter calls needed to establish object state—into explicit generation guidance. When coverage plateaus, it further applies coverage-driven *Backward Slicing* using the uncovered branch as the criterion, so that the prompt is rebuilt around the statements and cross-method dependencies that actually influence the target outcome. In this way, the two analyses serve a single branch-level alignment objective rather than acting as independent prompting add-ons.

The main contributions can be summarized as follows:

- We formulate branch-coverage failures in LLM-based test generation as a form of *cognitive misalignment*, caused by hidden cross-method preconditions and branch-irrelevant context.
- We propose COGPATH, an iterative generate–execute–repair framework that combines *Constraint-Hints* and coverage-driven *Backward Slicing* to make path-specific preconditions explicit and reduce irrelevant context.
- We evaluate COGPATH on 130 classes and 2,971 complex methods from 14 Defects4J projects. Under controlled backbone and context settings, COGPATH improves average line and branch coverage over the strongest baseline by 12.8 and 10.2 percentage points, respectively.
- We release COGPATH with its prompts, analysis components, and evaluation scripts to support replication and follow-up research.

II. MOTIVATION

Challenge 1: Underspecified Constraints (What to Satisfy). **Problem:** Branch reachability often depends on implicit object states established through inter-procedural interactions [5]. These constraints are not explicitly represented in the input, forcing the LLM to infer them from incomplete information.

Example: Consider the `addValue()` method from Apache Commons Math (Listing 1). The target branch depends on the condition `meanImpl != mean`, which can only be satisfied if a prior method modifies the internal state:

To reach this branch, `meanImpl` must be explicitly set to a non-null, non-default value before `addValue()` is called. An LLM provided only with the `addValue()` method body might generate:

This test is executable but fails to cover the branch because the critical precondition of `meanImpl` being initialized is unmet. The LLM lacks the context to “see” this inter-procedural requirement.

```
1 public void addValue(double value) {
2     ...
3     if (meanImpl != mean) { // target branch
4         meanImpl.increment(value);
5     }
6 }
```

Listing 1. Target branch with hidden cross-method dependency

```
1 SummaryStatistics stats = new
  SummaryStatistics();
2 stats.addValue(1.0);
```

Listing 2. LLM-generated test (constraint not satisfied)

By supplying the explicit constraint that `meanImpl` must differ from the default `mean`, the LLM successfully identifies the required precondition and generates a correct initialization sequence:

```
1 // Assume CustomMeanImpl is a valid
  implementation
2 SummaryStatistics stats = new
  SummaryStatistics();
3 stats.setMeanImpl(new CustomMeanImpl());
4 stats.addValue(1.0);
```

Listing 3. Correct test (constraint satisfied)

This example illustrates a general failure mode: when constraints are encoded as cross-method state transitions, they remain invisible unless explicitly surfaced, making them unlikely to be satisfied during generation. Challenge 1 addresses the case where critical information is absent from the provided context. A complementary problem arises when the information is present but obscured.

Challenge 2: Misaligned Context (What to Attend To). **Problem:** Even when all necessary conditions are locally available, they are often embedded within large methods containing substantial irrelevant logic. Providing the full method body introduces noise that is not aligned with the coverage objective.

Example: Consider the `collect()` method from `JDataBind`, where the target branch is guarded by a conjunction of three conditions:

```
1 if (checkJavaLangObject && (_intr != null) &&
   !methods.isEmpty()) { ... }
```

Listing 4. Target branch with multi-variable condition

Each condition originates from a different part of the method body. The full method interleaves the initialization of methods, the resolution of `checkJavaLangObject`, and several loops and nested branches entirely unrelated to the target guard. When exposed to the complete implementation, the LLM fails to identify that `_mixInResolver` is the critical dependency and generates a test that leaves `checkJavaLangObject` permanently false:

```
1 Collector collector=new Collector(intr, null);
  // _mixInResolver is null
2 collector.collect(typeFactory, tc, mainType,
  superTypes, primaryMixIn);
3 // checkJavaLangObject remains false; target
  branch never reached
```

Listing 5. LLM-generated test (branch not covered)

The bottleneck is that the LLM’s attention is diluted across the full method body. The critical dependency—that `checkJavaLangObject` is set to true only when `_mixInResolver` returns a non-null `mixIn`—is buried among dozens of unrelated statements. The model fails to trace this data-flow chain and leaves `_mixInResolver` as null.

By retaining only statements with data or control dependence on the target branch, the three guard dependencies become structurally visible, enabling the LLM to correctly configure the resolver and construct a valid test:

```

1 Map<MemberKey, MethodBuilder> methods = new
  LinkedHashMap<>();
2 _addMemberMethods(tc, mainType.getRawClass(),
  methods, primaryMixin);
3 // for (...) { ... } // DELETE: no dependence
  on target branch
4 boolean checkJavaLangObject = false;
5 if (_mixinResolver != null) {
6   Class<?> mixin =
  _mixinResolver.findMixinClassFor(Object.class);
7   if (mixin != null) {
8     _addMethodMixIns(...);
9     checkJavaLangObject = true; // sole setter
    of guard variable
10  }
11 }
12 if (checkJavaLangObject && (_intr != null) &&
  !methods.isEmpty()) { ... }

```

Listing 6. Branch-relevant slice

With this focused context, the three dependencies become structurally visible: `checkJavaLangObject` requires a non-null `mixin`, `_intr` is constructor-injected, and `methods` is populated by `_addMemberMethods`. The LLM now generates:

```

1 Collector collector = new Collector(intr,
  resolver);
2 // resolver returns non-null mixin ->
  checkJavaLangObject = true
3 // _addMemberMethods populates methods ->
  !methods.isEmpty()
4 collector.collect(typeFactory, tc, mainType,
  superTypes, primaryMixin);
5 // target branch covered

```

Listing 7. Test generated by our approach (branch covered)

This reflects a second failure mode: even when constraints are present in the source, they are not structurally aligned with the model’s attention, making precise reasoning difficult.

III. METHODOLOGY

A. Overview

COGPATH is a coverage-guided iterative framework for automated unit test generation, illustrated in Figure 1. Given a method under test, COGPATH first performs an initialization step that extracts project-level dependencies, resolves the required JUnit imports, and seeds the suite with a syntactically valid scaffolding test containing a trivial `assertTrue(true)` assertion, ensuring a compilable starting point for all subsequent iterations. It then constructs the CFG, enumerates feasible paths, and collects dynamic coverage feedback to identify uncovered branches before entering an iterative refinement loop.

Each iteration begins with path selection, where *Constraint-Hints* analyzes the selected uncovered path and injects explicit branch conditions as structured guidance into the generation prompt. When coverage improvement stagnates, *Backward Slicing* is activated to rebuild the prompt around source-grounded statements and setup actions likely to influence the target branch. The LLM then generates candidate tests against this focused context; passing tests are merged into the suite

while failing ones are forwarded to the repair phase. During repair, compiler diagnostics—including error messages and their associated line numbers—are incorporated into the repair prompt, allowing the LLM to precisely locate and correct the offending statements rather than re-generating the test from scratch. Coverage is re-measured at the end of each iteration to drive target selection in the next round. The loop terminates when the coverage target is reached, the iteration budget is exhausted, or coverage fails to improve for a configurable number of consecutive iterations.

B. Constraints-Hints

Constraints-Hints addresses the implicit precondition problem by decoupling constraint inference from test synthesis. Instead of requiring the LLM to simultaneously infer branch reachability and generate tests, we explicitly extract the conditions necessary to trigger a target branch based on its CFG path and provide them as structured guidance; the detailed prompt design is described in the supplementary material.

While symbolic execution can derive precise path conditions, it often struggles to scale to real-world programs due to path explosion, complex heap interactions, and challenges in modeling external libraries and reflection [1], [5]. Rather than performing exact constraint solving, we aim to extract *sufficient* conditions that characterize branch reachability.

Concretely, given an uncovered branch, we analyze its predicate and derive a set of constraint hints that describe the required program state (e.g., variable relations, nullness conditions, and key object properties). These hints are then embedded into the generation prompt, guiding the LLM to construct inputs and invocation sequences that satisfy the target condition.

By transforming implicit dependencies into explicit guidance, *Constraints-Hints* reduces the reasoning burden on the LLM and enables more reliable generation of tests that reach previously uncovered branches.

Path formulation. Let $P = \langle b_1, b_2, \dots, b_k \rangle$ denote the sequence of branch decisions along a target execution path π , where each predicate $b_i(\mathbf{x}) \in \{\text{true}, \text{false}\}$ is evaluated over the program input vector $\mathbf{x}$. Covering the path requires an input assignment $\mathbf{x}^*$ that satisfies all branch predicates along the path:

$$\mathbf{x}^* \in \mathcal{F}(\pi) = \{\mathbf{x} \mid b_1(\mathbf{x}) \wedge b_2(\mathbf{x}) \wedge \dots \wedge b_k(\mathbf{x})\}. \quad (1)$$

Rather than solving this constraint system symbolically—which can be computationally expensive and brittle under complex object states or indirect method calls—COGPATH employs an LLM-based constraint solver to approximate the feasible input space as a lightweight set of interpretable conditions $\mathcal{C}(\pi)$. The first uncovered branch along the path, denoted as b^* , is identified from the coverage report and provided to the solver as additional context, guiding the model toward the most immediate obstacle preventing the path from being executed.

Integration. The constraint-solving prompt is generated from a configurable template that incorporates the source code,

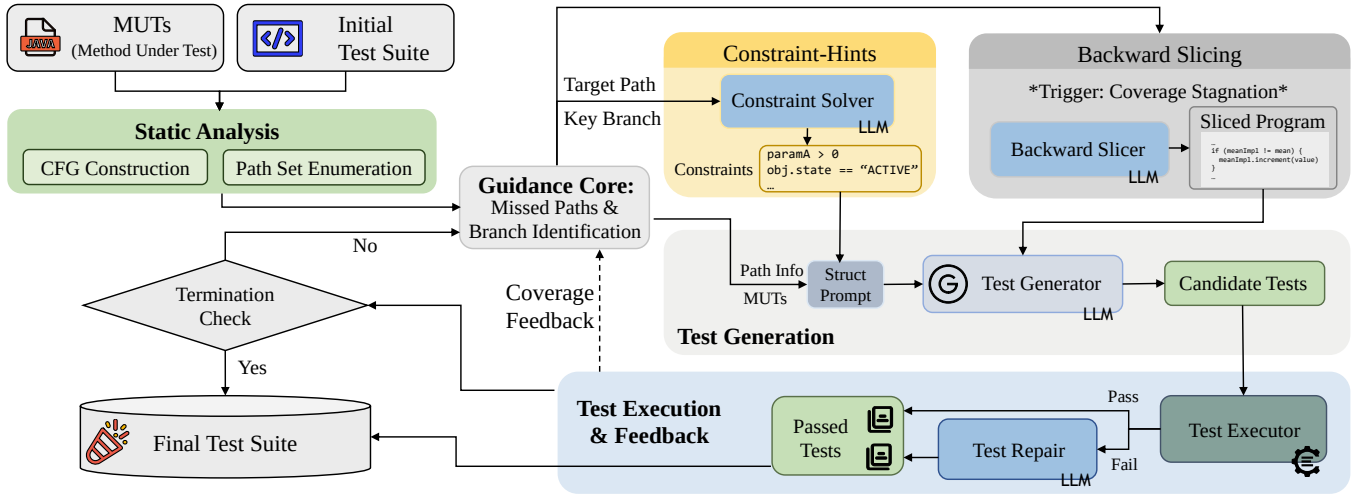


Fig. 1. Overview of the coverage-guided test generation framework.

the branch sequence of π , and the uncovered branch b^* . The solver produces $\mathcal{C}(\pi)$ as a list of conditions such as `paramA > 0` or `obj.status == "ACTIVE"`. These conditions are appended to the test generation prompt, transforming an opaque path coverage objective into a concrete specification of required inputs.

Example. Consider the `addValue` method, which contains three conditional branches dispatching to overridable statistical implementations. An existing test suite covers only the false branches of all three predicates, meaning `meanImpl`, `varianceImpl`, and `geoMeanImpl` all refer to their default implementations. The first uncovered branch is `(meanImpl != mean)`. The constraint solver identifies that satisfying this predicate requires replacing the default implementation before invoking `addValue`. This object-state precondition is expressed as a constraint in $\mathcal{C}(\pi)$, i.e., `c : meanImpl != mean`, which implies that a call such as `setMeanImpl(...)` must occur prior to invoking `addValue`. By making this requirement explicit, the test generation LLM can construct the correct setup sequence and cover the previously unreachable branch.

C. Branch-Aligned Context Slicing

Coverage improvement does not always progress smoothly: certain branches remain persistently uncovered even after multiple generate–execute–repair iterations. In such cases, continuing to expose the same full method body often repeats the same failure. COGPATH therefore performs *slicing-inspired branch context construction*: it rebuilds the prompt around source-grounded statements and setup actions that are likely to affect a specific uncovered branch.

This step is inspired by backward slicing, but it is not intended to replace classical SDG- or PDG-based slicing. Classical static slicers [18], [19], such as JavaSlicer [20], aim to compute sound dependence closures over complete and well-resolved programs. Our setting is different: the generation prompt is restricted to the declaring class source and imports,

while branch reachability may depend on object states established through setters, factory methods, or framework initialization logic. We therefore use slicing as a prompt-alignment operation rather than as a formal dependence-analysis claim. The produced context is approximate, source-grounded, and validated only through downstream compilation, execution, and coverage feedback.

Branch criterion. Let b^* denote the first uncovered branch on a selected path, with predicate P located at source line ℓ in the focal method M . We define the branch-context criterion as:

$$C = \langle \ell, P, V(P) \rangle, \quad (2)$$

where $V(P)$ contains the variables, fields, and receiver objects referenced by the predicate. When P invokes helper methods or accesses object fields, $V(P)$ is expanded with receiver state and field accesses that are visible within the declaring class. This criterion makes the target of context reduction explicit: the context is constructed for a concrete uncovered branch, not for the method as a whole.

Context extraction. Given C , the slicer LLM receives the declaring class source with line numbers, the focal method, the target predicate and line number, and the signatures or bodies of methods in the declaring class that are referenced by the focal method. The model is prompted to return a structured tuple:

$$\langle stmts, states, setups, rationale \rangle,$$

where *stmts* are source-grounded statements that may data- or control-influence P , *states* describe object states required at ℓ , *setups* list candidate calls such as setters or factory methods needed to establish those states, and *rationale* briefly explains the dependency chain.

To reduce hallucination, COGPATH validates the returned statements against the numbered source code and discards entries whose line numbers or text do not match the declaring class. The resulting context is therefore not assumed to be

sound, complete, or minimal; it is a branch-focused prompt artifact used to guide the next generation attempt.

Use in test generation. The validated tuple is supplied to the test-generation prompt as focused branch context. Generated tests are compiled, executed, and measured using the same validation pipeline as regular candidates. A slice affects the final suite only through tests that compile, pass, and improve or preserve coverage; incorrect or incomplete contexts are filtered by execution feedback rather than being accepted as program-analysis results.

D. Iterative Workflow of COGPATH

The complete workflow of COGPATH is formalized in Algorithm 1. The framework accepts the source file and an initial test suite as input, and iteratively generates, validates, and repairs tests until a coverage target is met or a termination condition is triggered.

Initialization. The framework extracts test-scope dependencies, constructs the CFG and method dictionary from the source file, deriving branch sequences and inter-procedural dependency information to bootstrap Constraint-Hints and Backward Slicing respectively. Baseline coverage is then measured on the initial suite (lines 1–4), providing the missed-branch information that drives subsequent path selection.

Path selection with Constraints-Hints. At each non-initial iteration, SELECTUNCOVEREDPATHS identifies candidate paths from the CFG that intersect with missed lines or branches, weighted by both coverage impact and visit history *pathHistory* (line 8). For each selected path π , the constraint solver derives $\mathcal{C}(\pi)$ by analyzing the branch sequence up to the first uncovered predicate b^* , and these conditions are embedded into the generation prompt (lines 9–13).

Backward slicing. When $noIncr \geq \beta \cdot \Lambda$, indicating a coverage plateau, the backward slicer is invoked on the remaining missed code segments. It returns a source-grounded branch context—relevant statements, required object states, and setup actions—which is used to construct an additional batch of targeted tests $\mathcal{T}_s$ (lines 16–19).

Validation and repair. Each generated test is individually inserted into the suite and executed. Failing tests are collected with their error messages and forwarded to the repair phase, which re-invokes the LLM with failure feedback to produce corrected versions. Successfully validated repaired tests are also incorporated into the suite (lines 21–32).

Termination. The loop exits when the coverage target τ is reached, the maximum iteration count *maxIter* is exhausted, or coverage fails to improve for Λ consecutive iterations.

IV. EXPERIMENTS

We design our experiment to answer the following three research questions (RQs):

RQ1: Effectiveness How does COGPATH perform compared to baselines in terms of line and branch coverage, especially on complex, logic-heavy methods?

RQ2: Ablation Study What are the individual contributions of Backward Slicing and Constraint Hints to the overall

Algorithm 1: Iterative Test Generation with Constraints-Hints and Backward Slicing

```

Input: src: source file, suite: initial test suite (possibly empty)
Output: Augmented test suite with improved coverage
deps  $\leftarrow$  EXTRACTDEPS();
cfg, methodDict  $\leftarrow$  BUILDCFG(src);
noIncr  $\leftarrow$  0, iter  $\leftarrow$  0;
cov, missed  $\leftarrow$  RUNCOVERAGE(suite);
while iter < maxIter and noIncr <  $\Lambda$  and cov <  $\tau$  do
    // Path selection and prompt construction
    if cov = 0 or iter = 0 then
        prompt  $\leftarrow$ 
            BUILDBASELINEPROMPT(src, suite, deps);
        ; // baseline prompt
    else
         $\Pi \leftarrow$  SELECTPATHS(cfg, methodDict, missed);
         $\mathcal{C} \leftarrow \emptyset$ ;
        foreach path  $\pi \in \Pi$  do
             $b^* \leftarrow$  FIRSTUNCOVEREDBRANCH( $\pi, missed$ );
             $\mathcal{C}(\pi) \leftarrow$  CONSTRAINTSOLVER(src,  $\pi, b^*$ );
            ; // Constraints-Hints
        end
        prompt  $\leftarrow$  BUILDPROMPT(src, suite, deps,  $\Pi, \mathcal{C}$ );
    end
    // Test generation
     $\mathcal{T} \leftarrow$  GENERATETESTS(prompt);
    // Backward slicing for stubborn uncovered paths
    if noIncr  $\geq \beta \cdot \Lambda$  then
        slices  $\leftarrow$  BACKWARDSLICER(src, missed);
         $\mathcal{T} \leftarrow$ 
             $\mathcal{T} \cup$  GENERATEFROMSLICES(slices, suite, deps);
    end
    // Validation and repair
    failed  $\leftarrow \{\}$ ;
    foreach t  $\in \mathcal{T}$  do
        if VALIDATETEST(t, suite) then suite  $\leftarrow$  suite  $\cup \{t\}$ ;
        else failed  $\leftarrow$  failed  $\cup \{t\}$ ;
    end
    if failed  $\neq \emptyset$  then
        suite  $\leftarrow$  suite  $\cup$ 
            REPAIRANDVALIDATE(failed, src, suite, deps);
    end
    // Coverage update and termination check
    (cov', missed')  $\leftarrow$  RUNCOVERAGE(suite);
    if cov' > cov then noIncr  $\leftarrow$  0;
    else noIncr  $\leftarrow$  noIncr + 1;
    cov  $\leftarrow$  cov', missed  $\leftarrow$  missed', iter  $\leftarrow$  iter + 1;
end
return suite;

```

coverage of COGPATH, and how does the iterative generation loop affect the branch coverage?

RQ3: Generality Is COGPATH's effectiveness robust across different underlying large language models, and how do their performances vary?

A. Experimental Setup

Dataset. To evaluate the effectiveness of COGPATH in generating unit tests for complex methods, we utilize real-

TABLE I
DATASET METRICS WITH #MUTs

Identifier	Project	Version	#C	#MUTs	CC_{max}	CC_{min}	CC_{med}
Cli	Cli-40f	1.5-SNAP	2	31	11	11	11
Codec	Codec-18f	1.11-SNAP	7	78	31	11	13
Collections	Collections-28f	4.2-SNAP	5	218	32	11	16
Compress	Compress-47f	1.16-SNAP	9	65	22	11	13
Csv	Csv-16f	1.6-SNAP	3	72	25	11	14
Gson	Gson-16f	2.8.2-SNAP	4	75	37	11	22
JCore	JacksonCore-26f	2.10.0-SNAP	9	161	30	11	14
JDataBind	JacksonDataBind-112f	2.9.9	9	371	22	11	15
JXml	JacksonXml-5f	2.9.6-SNAP	4	118	29	12	22
Jsoup	Jsoup-93f	1.12.2-SNAP	8	171	22	11	12
JXPath	JXPath-22f	1.4-SNAP	12	115	32	11	13
Lang	Lang-4f	3.2-SNAP	17	586	30	11	16
Math	Math-2f	3.3-SNAP	30	452	31	11	14
Time	Time-13f	2.1	11	458	30	11	17
Total/Avg			130	2971	27.4	11.1	15.1

world subjects from the Defects4J (v2.0.1) benchmark [21]. We selected the latest fixed version of each subject program to ensure compatibility with modern build environments. Following prior work [12], we excluded projects that do not use Maven or rely on deprecated dependencies (i.e., Chart, Mockito, and Closure). The remaining 14 subjects are compatible with Java 8+, Maven (v3.6.3), and JUnit (v4.13.2). To specifically assess performance on logic-heavy code, we focus on complex classes rather than entire projects. We identified non-abstract, public classes containing at least one method with Cyclomatic Complexity (CC) > 10 [22]. The statistics of our evaluation dataset are summarized in Table I.

Evaluation Metrics. To evaluate the effectiveness of the generated unit tests, we primarily focus on code coverage, which is the standard metric in automated test generation research [2], [4]. We report two specific metrics:

- **Line Coverage (LC):** The ratio of executable lines in the target class that are exercised by the generated test suite. It provides a baseline measure of how much of the implementation has been reached.
- **Branch Coverage (BC):** The ratio of executed decision outcomes (e.g., the true and false paths of an *if* statement) to the total number of possible branches. Given our focus on methods with high Cyclomatic Complexity ($CC > 10$), branch coverage is our primary metric.

All coverage data in our experiments are collected using the JaCoCo library, ensuring consistent and reproducible measurements across all subjects and baselines.

Baselines. Prior studies, including SymPrompt and HITS, suggest that LLM-based approaches can outperform SBST tools such as EvoSuite under comparable experimental settings. Therefore, we focus on comparisons with state-of-the-art LLM-based baselines. CoverUp targets Python and operates at line-level granularity, making direct comparison on our Java branch-coverage benchmark infeasible. Therefore, we compare COGPATH against three representative LLM-based unit test generation frameworks that capture diverse design paradigms in the field, plus a Vanilla Prompt control to isolate framework-

level gains from the backbone LLM’s intrinsic capability:

- **Vanilla Prompt:** A plain prompt that asks the backbone LLM to generate a JUnit test suite with 100% branch coverage from the focal class source, without framework support.
- **SymPrompt [11]:** A prompt-engineering approach that encodes symbolic execution concepts into structured prompts. It extracts branch conditions from CFGs and constructs path-oriented prompts to guide the LLM in generating inputs that satisfy specific execution paths.
- **HITS [10]:** A hierarchical prompting framework that decomposes complex methods into smaller code segments before test generation. It adopts a self-healing strategy that iteratively repairs failing tests to improve compilability and coverage.¹
- **Panta [12]:** A hybrid framework that combines static CFG analysis with dynamic coverage feedback to locate uncovered paths. It constructs context-aware prompts that expose intra-class data dependencies to guide the LLM toward uncovered branches.

For fair comparison, all methods use the same backbone model as COGPATH in each experiment. In RQ1 and RQ2, we use Qwen3-Coder-30B-A3B (QC3-30B) deployed locally on an NVIDIA H800 GPU (80 GB), and repeat each experiment three times with averaged results. RQ3 additionally evaluates Qwen3.5-397B-A17B, GPT-5.4-Mini, and DeepSeek-V3.1 to assess generalizability. For each MUT, all methods access only the declaring class source code and imports, excluding project-level dependencies and configurations. All models use temperature 0.2 and top- p 1.0. COGPATH is configured with $maxIter = 20$, $\Lambda = 3$, $\beta = 0.6$, target coverage $\tau = 100.00\%$, and starts from an empty test suite.

¹Due to missing components in the official HITS repository, we re-implemented its workflow by strictly following the prompt templates described in the paper and the procedural logic of available open-source fragments.

TABLE II
COVERAGE ACROSS APPROACHES

Project	CC_{avg}	Vanilla Prompt		SymPrompt		HITS		Panta		COGPATH	
		LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	11.00	36.81	31.99	17.93	9.02	73.50	59.58	63.11	37.98	84.32	70.73
Codec	16.14	24.58	21.96	29.68	24.96	65.73	60.25	60.53	54.82	76.97	69.60
Collections	18.60	8.38	6.72	32.63	32.42	68.23	70.00	33.44	32.40	40.46	34.63
Compress	15.33	7.49	4.31	11.84	11.34	31.64	25.24	34.36	28.45	36.09	29.05
Csv	16.67	25.22	14.46	37.69	23.79	28.95	14.73	50.98	37.99	62.27	52.29
Gson	23.25	12.78	2.32	13.16	3.59	31.54	21.25	21.81	15.20	63.82	41.24
JCore	16.33	19.44	15.13	23.62	21.44	19.85	18.51	26.24	21.97	41.72	35.54
JDatabind	15.33	7.95	4.98	9.99	6.06	18.33	14.44	18.17	14.34	23.35	17.85
JXml	21.50	19.13	14.86	17.45	18.56	18.59	18.84	31.89	23.55	20.61	17.08
Jsoup	14.13	27.30	17.94	24.72	19.13	27.82	17.36	49.80	39.25	66.43	52.55
JxPath	15.50	17.66	12.19	12.01	9.48	23.11	22.76	34.10	29.35	42.28	35.84
Lang	16.88	21.35	17.10	43.98	40.42	62.82	59.77	39.26	36.61	59.21	52.14
Math	16.47	25.27	20.83	38.55	33.70	30.87	26.38	46.40	39.16	59.60	54.02
Time	17.36	31.42	23.85	53.37	42.06	63.32	50.23	46.74	41.42	66.79	59.25
Average		20.34	14.90	26.19	21.14	40.31	34.24	39.77	32.32	53.14	44.41

B. RQ1: Effectiveness

To answer RQ1, we compare COGPATH against three state-of-the-art baselines on the selected dataset. Results are summarized in Table II.

1) *Overall Performance*: As shown in Table II, COGPATH achieves the highest average LC and BC across all projects. Specifically, it attains an average BC of 44.41%, outperforming SymPrompt (21.14%), HITS (34.24%), and Panta (32.32%). Same trends hold for LC, where COGPATH achieves 53.14% versus SymPrompt (26.19%), HITS (40.31%), and Panta (39.77%). The Vanilla Prompt control reaches only 14.90% BC—lower than every structured baseline, including SymPrompt—and trails COGPATH by **29.5%**. Because this comparison uses the identical backbone LLM, the gap is consistent with the hypothesis that logic-heavy methods ($CC > 10$) require more structured guidance than plain prompting provides. It also reduces the likelihood that the observed improvement is explained solely by the backbone model, although additional diagnostics are needed to isolate the exact contribution of each guidance mechanism.

Gains over the two strongest baselines—HITS (**+10.2%**) and Panta (**+12.1%**)—suggest that jointly reducing irrelevant context and making implicit preconditions explicit can cover branches that remain difficult for iterative refinement and context-aware prompting alone. The margin widens substantially against SymPrompt (**+23.3%**), which is consistent with the need for branch-specific context construction when navigating logic-heavy control flows.

2) *Performance on High-Complexity Projects*: The advantages of COGPATH are most pronounced on projects with high cyclomatic complexity and deeper inter-method dependencies.

For instance, on Gson ($CC_{avg} = 23.25$, the highest in the benchmark), COGPATH achieves a BC of 41.24% while SymPrompt reaches only 3.59%; on JCore ($CC_{avg} = 16.33$), BC rises from 18.51% (HITS) to 35.54%; and on Jsoup (CC_{avg}

= 14.13), BC improves from 17.36% (HITS) to 52.55%. These project-level gains are consistent with the intended roles of backward slicing and constraint hints: reducing branch-irrelevant context and surfacing implicit preconditions in code with dense branching logic and cross-method dependencies.

Answer to RQ1. COGPATH outperforms all three baselines on average, achieving 44.41% BC (+10.2%–23.3% over baselines), with the largest gains on high-CC, dependency-rich projects (e.g., +20% on Gson, +14% on JCore). The Vanilla Prompt control (14.90% BC) provides evidence that structured framework guidance contributes to the improvement beyond the backbone LLM alone.

C. RQ2: Ablation Study

1) *Contribution of BS and CS*: We construct three variants—**w/o BS** (removing backward slicing), **w/o CS** (removing constraint hints), and **w/o CS+BS** (removing both)—all sharing the same iterative loop limit ($\text{maxIter}=20$) and evaluated under QC3-30B. Results are assessed via Table III.

Table III shows that both components contribute independently: removing BS alone drops average BC by 5.94 pp (44.41% $\rightarrow$ 38.47%), removing CS alone by 4.62 pp (44.41% $\rightarrow$ 39.79%), and removing both by 7.84 pp (44.41% $\rightarrow$ 36.57%). The joint degradation falls below the sum of individual losses (10.56 pp), a subadditivity that reflects partial functional overlap—CS not only makes cross-method preconditions explicit but also implicitly focuses the LLM on branch-relevant predicates, partially mirroring BS’s context-reduction role—though neither component can fully substitute for the other. The same degradation ordering holds for LC (BS removal: -6.63 pp; CS removal: -6.56 pp; both: -9.70 pp).

Yet these averages mask substantial per-project variation: BS and CS address structurally distinct bottlenecks, making them complementary rather than redundant. BS contributes most where method bodies are dominated by branch-irrelevant statements: Csv is a highly configurable, multi-dialect parser

TABLE III
ABLATION STUDY OF COGPATH COMPONENTS

Project	CC_{avg}	CogPath		w/o CS		w/o BS		w/o CS+BS	
		LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	11.00	84.32	70.73	84.74	68.75	68.69	57.68	72.62	56.54
Codec	16.14	76.97	69.60	54.86	50.68	66.77	57.94	66.34	56.53
Collections	18.60	40.46	34.63	39.51	38.59	32.42	36.60	37.43	34.45
Compress	15.33	36.09	29.05	31.22	28.58	25.43	19.04	23.45	18.11
Csv	16.67	62.27	52.29	46.18	36.38	36.47	24.80	60.54	47.07
Gson	23.25	63.82	41.24	43.94	27.73	49.40	31.85	40.92	26.52
JCore	16.33	41.72	35.54	30.58	27.31	45.39	39.20	22.97	21.07
JDatabind	15.33	23.35	17.85	12.44	9.33	33.74	26.55	19.59	15.54
JXml	21.50	20.61	17.08	23.80	18.61	35.30	26.84	26.79	23.08
Jsoup	14.13	69.11	53.76	61.48	46.64	69.11	53.76	59.00	47.11
JxPath	15.50	42.28	35.84	47.56	41.96	28.06	22.35	34.94	31.44
Lang	16.88	59.21	52.14	66.05	63.04	56.84	51.38	57.12	53.91
Math	16.47	59.60	54.02	63.42	56.61	38.60	32.66	43.23	39.90
Time	17.36	66.79	59.25	46.29	42.81	64.99	57.89	43.21	40.65
Average		53.14	44.41	46.58	39.79	46.51	38.47	43.44	36.57

whose methods interleave handling for delimiters, quotes, escape sequences, and EOF within a single control flow. Backward slicing strips these dialect-processing branches that are irrelevant to the target predicate, letting the LLM focus on the right conditions; without it, the interference from unrelated branches collapses BC from 52.29% to 24.80% (−27.49 %). CS contributes most where branch reachability is governed by cross-call preconditions rather than local logic: in *JDatabind*, reaching many branches requires a specific sequence of configuration API calls to establish the correct mapper or reader/writer state—constraints that are invisible within the focal method body alone. CS makes these preconditions explicit; without it, the LLM readily produces syntactically valid tests that nonetheless fail to satisfy the required object state, reducing BC from 17.85% to 9.33% (−8.52 %). In short, BS addresses context noise while CS addresses hidden state dependencies—distinct bottlenecks that neither component can resolve for the other.

This benefit, however, is not universal: on *JCore* and *JXml*, removing BS raises coverage, pointing to two related failure modes. First, many branch predicates in these projects are feature-flag checks whose controlling field is set via a builder chain at construction time, not passed as a method argument. BS correctly traces the data flow to the field, but the full assignment path spans multiple factory-method layers; the resulting slice is semantically incomplete rather than merely short, and the LLM is left with worse context than the unsliced method body. Second, *JXml* compounds this with a two-layer configuration stack—a Woodstox *WstxInputFactory* beneath Jackson’s *XmlMapper*—so BS structurally misses the lower layer’s dependencies from the start, not merely truncates them.

2) *Improvement of Iterative Framework*: Beyond final coverage, we examine whether the iterative loop itself accelerates progress. We ran all four configurations (COGPATH and its

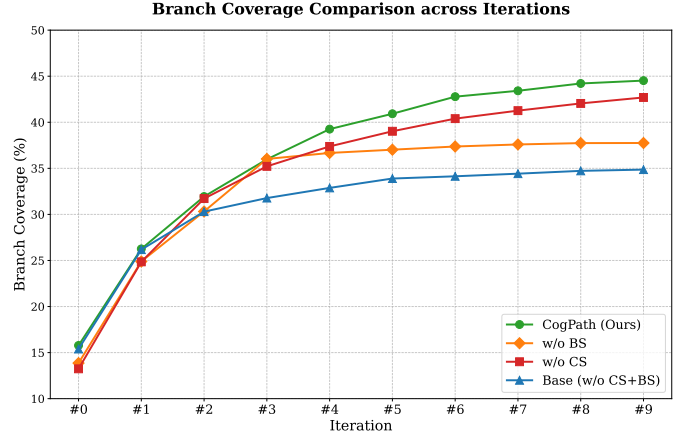


Fig. 2. Branch coverage improvement over iterations for COGPATH and its three ablated variants.

three ablated variants) under QC3-30B, recording average BC at each step; the resulting growth curves are shown in Figure 2. For clarity, we plot the first 10 iterations only, because all curves become nearly flat afterward and additional rounds yield negligible gains.

Across the entire range, COGPATH remains the top-performing configuration at every iteration, showing both the fastest early-stage growth and the highest eventual plateau. In contrast, w/o BS and w/o CS rise more slowly and saturate at lower coverage levels, while w/o CS+BS plateaus the earliest with the lowest ceiling. This three-way pattern—faster rise, higher ceiling, and later plateau—indicates that BS and CS improve iterative testing in two complementary ways: they increase the effectiveness of each generation round and they postpone premature saturation.

Answer to RQ2. Both backward slicing (BS) and constraint-hints (CS) make independent, complementary contributions

and neither component is dispensable: BS eliminates context noise (−5.94 % on average), while CS makes implicit cross-method preconditions explicit (−4.62 % on average). The iterative loop further amplifies these gains: COGPATH leads at every iteration with the fastest early growth and the latest saturation.

D. RQ3: Model Generalizability

To evaluate model generalizability, we run COGPATH under four backbone models—QC3-30B (used in RQ1 and RQ2), GPT-5.4-Mini, QW3.5-397B, and DS-V3.1—using the same experimental configuration as RQ1. Results are summarized in Table IV.

1) *Overall trends.*: Scaling up the backbone model brings substantial coverage improvements across all 14 projects. While QC3-30B achieves an average BC of 44.41%, the three large-scale models reach 79.76% (GPT-5.4-Mini), 80.36% (DS-V3.1), and 83.47% (QW3.5-397B)—gains of 35–39 % over QC3-30B. Among them, QW3.5-397B achieves the highest overall BC, consistent with its leading performance on code-oriented benchmarks documented in its technical report [23]. Notably, even QC3-30B already surpasses every RQ1 baseline (best: HITS at 34.24%), suggesting that COGPATH’s framework-level guidance is beneficial even with a smaller backbone model.

2) *Where model scale matters most.*: The capacity-driven gains are pronounced on projects that demand dense conditional reasoning. On JDataBind, QC3-30B reaches only 17.85% BC while large models achieve 68–77%. Cross-referencing with the RQ2 ablation sharpens this picture: removing CS costs 8.52% on JDataBind, while removing BS improves coverage by 8.70 %. Together, these results indicate that JDataBind’s dense cross-method initialization logic depends critically on explicit constraint guidance rather than code-context breadth. It is precisely this constraint reasoning and following capability—translating structured hints into correct multi-step object state construction—that scales with model size, explaining the pronounced gap between QC3-30B and the larger models on this project. By contrast, on structurally simpler projects such as Cli and Time, the gap narrows to under 20 %, where COGPATH’s structural guidance already suffices for QC3-30B to handle the less demanding conditional logic.

3) *Structural bottlenecks.*: Despite the strong overall gains, JCore and JXml remain below the large-model average of ~81% regardless of backbone—best BC of 76.77% and 74.20%, respectively. As identified in RQ2, these projects share a framework-level bottleneck: their branch predicates are governed by builder-chain configuration state that spans multiple factory-method layers, causing BS to produce semantically incomplete slices that are worse than the unsliced method body. Scaling up model capacity cannot compensate for structurally deficient input, which is why the ceiling persists even under the strongest models. The consequence is also visible in higher inter-model variance on these two

projects (e.g., JCore: QW3.5-397B=76.77% vs. GPT-5.4-Mini=57.83%), suggesting that when COGPATH’s structural guidance is compromised, performance may fall back on model-specific code reasoning rather than shared framework scaffolding. Extending backward slicing to handle cross-layer builder chains is therefore the primary direction for future refinement.

Answer to RQ3. COGPATH performs robustly across all four backbone LLMs, with stronger models consistently yielding higher coverage (79.76–83.47% avg. branch coverage for large models). On high-complexity projects with deep inter-method dependencies, improved model reasoning ability further amplifies this advantage.

V. DISCUSSION

Inference Token Cost. To present cost in a deployment-relevant form, we report per-class cost under the four LLM backbones used in RQ3: QC3-30B (0.079\$/class), GPT-5.4-Mini (0.44\$/class), QW3.5-397B (0.78\$/class), and DS-V3.1 (0.2\$/class). Across all four settings, the per-class cost remains low in absolute terms, indicating that the framework-level overhead of COGPATH is operationally affordable.

Runtime Overhead. Across all 130 classes using QC3-30B as the backbone, COGPATH averages 692 s/class, incurring +47% overhead over SymPrompt (470 s/class) and +13% over HITS (610 s/class), while running 4% faster than Panta (720 s/class). Against these time differentials, COGPATH achieves a BC of 44.41% versus 21.14%, 34.24%, and 32.32% for the three baselines—relative improvements of +110%, +30%, and +37% respectively. The overhead is thus proportionate to the coverage gains in every comparison. Of the 692 s, ≈1.2 s is attributable to static preprocessing, which scales with the inter-procedural dependency graph and is already included in the figures above; the remainder is LLM inference time.

Threats to Validity. The effectiveness of COGPATH depends on the capabilities of the underlying LLM, and results may vary as more advanced models are introduced; we mitigate this by evaluating across four models spanning a range of scales and architectures (RQ3). Data contamination is a known concern in LLM-based software engineering research, as evaluated models may have encountered Defects4J code during pretraining; to reduce its effect, all methods are evaluated on the same projects with the same backbone model, so reported results reflect relative improvements between approaches rather than absolute figures. Our evaluation is further limited to Java projects from the Defects4J benchmark, which may affect generalizability to other languages or ecosystems, though the underlying techniques are language-agnostic in principle. Finally, our dataset is restricted to methods with $CC > 10$, as COGPATH specifically targets logic-heavy code; performance on lower-complexity methods has not been assessed, though COGPATH is intended to complement rather than replace existing approaches that already handle such cases well.

TABLE IV
MODEL GENERALIZATION ACROSS DIFFERENT LLMs

Project	#C	CC	QC3-30B		GPT-5.4-Mini		QW3.5-397B		DS-V3.1	
			LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	2	11.00	84.32	70.73	97.86	86.18	98.82	92.63	98.82	91.93
Codec	7	16.14	76.97	69.60	89.78	82.61	98.32	95.30	93.60	88.46
Collections	5	18.60	40.46	34.63	98.31	94.14	98.16	94.43	98.02	94.20
Compress	9	15.33	36.09	29.05	87.15	80.29	86.92	80.90	81.32	76.04
Csv	3	16.67	62.27	52.29	87.42	77.76	95.71	87.42	85.70	77.53
Gson	4	23.25	63.82	41.24	90.26	83.06	92.88	87.97	92.13	82.86
JCore	9	16.33	41.72	35.54	63.71	57.83	83.25	76.77	66.62	57.76
JDatabind	9	15.33	23.35	17.85	76.16	68.78	74.93	69.81	84.06	77.44
JXml	4	21.50	20.61	17.08	73.41	74.20	62.20	63.65	66.06	65.22
Jsoup	8	14.13	66.43	52.55	93.37	82.96	95.87	87.49	87.15	79.98
JXPath	12	15.50	42.28	35.84	81.72	75.11	71.30	66.22	76.89	73.21
Lang	17	16.88	59.21	52.14	90.65	84.32	91.72	87.44	88.21	85.02
Math	30	16.47	59.60	54.02	88.89	81.82	92.96	87.47	91.69	85.41
Time	11	17.36	66.79	59.25	94.18	87.63	96.94	91.08	94.71	89.92
Average			53.14	44.41	86.63	79.76	88.57	83.47	86.07	80.36

VI. RELATED WORK

A. Automated Unit Test Generation

Early techniques such as Randoop [4] and EvoSuite [2] drive test generation by optimizing structural coverage objectives. EvoSuite applies evolutionary algorithms over whole test suites to maximize branch coverage and automatically infers regression assertions. While effective at exploring input spaces, SBST methods fundamentally operate on syntactic signals and struggle with semantic preconditions that require understanding object states or inter-procedural call sequences [3].

Neural sequence-to-sequence models offered a first bridge between code understanding and test synthesis. ATLAS [24] frames test generation as a translation task, training an encoder-decoder network to predict assert statements from focal methods. AthenaTest [25] extends this direction by fine-tuning pre-trained models on large method-test corpora to automate full JUnit test synthesis. These deep learning approaches demonstrate that neural models can capture surface-level code patterns, but their capacity is fundamentally bounded by the training distribution: they cannot reliably generalize to the combinatorial space of runtime behaviors or reason about execution semantics [26].

The advent of large language models enables open-ended test synthesis grounded in broad code understanding. Early work such as ChatUniTest [7], CodaMosa [27] demonstrates the potential of LLM-based generation, but also exposes a key limitation: without structured guidance, models tend to produce superficially plausible tests that fail to reach complex branches.

A first wave of follow-up work addresses this gap by incorporating execution signals into the generation loop. CoverUp [28] iteratively targets untested code regions using dynamic coverage reports, directing the model’s attention toward uncovered lines, while YATE [9] further repairs failing tests to improve validity. FailMapper [29] integrates a Failure-

Aware Monte Carlo Tree Search algorithm [30] with LLMs to intelligently explore test inputs targeting these failure scenarios.

A second wave enriches the model’s input with structural program information. SymPrompt [11] extracts branch conditions from control flow graphs (CFGs) and embeds them as structured prompts. HITS [10] decomposes complex methods into smaller slices to reduce cognitive load, improving syntactic validity. Panta [12] combines static CFG analysis with dynamic coverage feedback to identify and target uncovered paths. JUnitGenie [13] further achieves strict path sensitivity by constructing minimal, path-specific contexts using type and data-dependency information. PALM [31] similarly embeds intra-procedural branch conditions extracted from the method-level CFG into path-specific prompts, but does not address the inter-procedural object-state dependencies that Constraint-Hints is designed to surface. LSPRag [32] leverages language server protocols to retrieve cross-file dependency context, augmenting the model’s view beyond the focal method. These advances substantially reduce the structural mismatch between model input and the coverage objective. However, a key limitation remains: branches whose reachability depends on cross-method object states-preconditions established by prior setter or initializer calls—are still difficult to capture, as such dependencies are not explicit in the focal method. In addition, even with path-level guidance, irrelevant statements in the full method context introduce substantial noise, degrading model effectiveness as complexity grows [16], [17]. Our work targets these limitations by making implicit preconditions explicit and focusing the model on branch-relevant dependencies.

B. Program Slicing

Program slicing [18], [33] extracts the subset of statements that influence a particular variable or program point. Classical static slicers identify data and control dependencies via program dependence graphs [19]; JavaSlicer [20] extends

this to Java programs by constructing precise inter-procedural dependence graphs over bytecode. These techniques have been widely applied in debugging, program comprehension, and regression testing. However, classical static slicers require complete, parsable code: tools such as Joern fail on incomplete snippets due to missing type declarations, unresolved imports, and references to external libraries. NSSlicer [34] addresses this limitation by leveraging a pre-trained language model to learn fine-grained variable–statement dependency patterns, enabling slicing of both complete and partial Java programs where traditional analysis is brittle.

More recently, Shahandashti et al. [35] demonstrate that LLMs can perform program slicing directly from source code without dedicated analysis infrastructure, achieving results competitive with specialized neural slicing tools. In particular, GPT-4o under one-shot with chain-of-thought prompting reaches an Accuracy-D of 60.84%, slightly surpassing NS-Slicer (CodeBERT [36]) at 60.52%, while also generating exact matches that NS-Slicer entirely fails to produce.

HITS [10] applies a form of method decomposition to reduce LLM input complexity. However, its decomposition is driven by structural heuristics rather than data and control dependencies, and is not targeted toward any specific uncovered branch. For high-complexity methods, the number of generated segments can grow substantially, incurring token overhead regardless of coverage impact.

In contrast, COGPATH performs coverage-driven, slicing-inspired context construction: it starts from a specific uncovered branch and builds a source-grounded prompt context around statements, object states, and setup actions likely to influence that branch. Unlike HITS’s structural decomposition, this step is triggered only when coverage stalls and is evaluated through downstream compilation, execution, and coverage feedback rather than claimed as a sound static slice.

VII. CONCLUSION AND FUTURE WORK

We presented COGPATH, an LLM-driven framework for automated unit test generation that addresses two complementary dimensions of cognitive misalignment: Constraint-Hints makes implicit cross-method preconditions explicit, while coverage-driven Backward Slicing eliminates irrelevant context to concentrate the model’s attention on branch-relevant dependencies. Evaluated on 130 classes and 2,971 complex methods from 14 real-world Java projects, COGPATH achieves 44.41% average branch coverage, outperforming state-of-the-art baselines by 10.2pp and generalizing consistently across LLMs of varying capacity.

Future work will pursue two directions. First, we plan to extend the framework with adaptive slice granularity selection and broader constraint guidance for complex object-oriented initialization patterns. Second, to assess how COGPATH’s structural guidance compares with general-purpose agentic test generation, we will integrate its ConstraintSolver and BackwardSlicer into OpenHands and evaluate the resulting agent on the same benchmark under the same backbone model. To ensure a like-for-like comparison, the agent will operate

under the identical information budget used in this paper—restricted to the declaring class source and import statements, with no access to external documentation, web search, or project-level dependencies—so that any difference in coverage and per-class cost is attributable to the agentic loop rather than to an enlarged information frontier.

VIII. DATA AVAILABILITY STATEMENT

The replication package, including the source code of COGPATH and evaluation scripts, is available for review at <https://doi.org/10.5281/zenodo.19249603>

REFERENCES

- [1] C. Cadar, D. Dunbar, D. R. Engler *et al.*, “Klee: unassisted and automatic generation of high-coverage tests for complex systems programs.” in *OSDI*, vol. 8, 2008, pp. 209–224.
- [2] G. Fraser and A. Arcuri, “Evosuite: automatic test suite generation for object-oriented software,” in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, ser. ESEC/FSE ’11. New York, NY, USA: Association for Computing Machinery, 2011, pp. 416–419. [Online]. Available: <https://doi.org/10.1145/2025113.2025179>
- [3] P. McMinn, “Search-based software testing: Past, present and future,” in *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*, 2011, pp. 153–163.
- [4] C. Pacheco and M. D. Ernst, “Randoop: feedback-directed random testing for java,” in *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion*, ser. OOPSLA ’07. New York, NY, USA: Association for Computing Machinery, 2007, pp. 815–816. [Online]. Available: <https://doi.org/10.1145/1297846.1297902>
- [5] R. Baldoni, E. Coppa, D. C. D’elia, C. Demetrescu, and I. Finocchi, “A survey of symbolic execution techniques,” *ACM Computing Surveys (CSUR)*, vol. 51, no. 3, pp. 1–39, 2018.
- [6] Z. Yuan, M. Liu, S. Ding, K. Wang, Y. Chen, X. Peng, and Y. Lou, “Evaluating and improving chatgpt for unit test generation,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1703–1726, 2024.
- [7] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin. ChatUniTest: A Framework for LLM-Based Test Generation. [Online]. Available: <http://arxiv.org/abs/2305.04764>
- [8] B. Chu, Y. Feng, K. Liu, Z. Nan, Z. Guo, and B. Xu, “Large Language Models for Unit Test Generation: Achievements, Challenges, and the Road Ahead,” Nov. 2025.
- [9] M. Konstantinou, R. Degiovanni, J. M. Zhang, M. Harman, and M. Papadakis, “Yate: The role of test repair in llm-based unit test generation,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.18316>
- [10] Z. Wang, K. Liu, G. Li, and Z. Jin, “HITS: High-coverage LLM-based Unit Test Generation via Method Slicing,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, pp. 1258–1268. [Online]. Available: <https://dl.acm.org/doi/10.1145/3691620.3695501>
- [11] G. Ryan, S. Jain, M. Shang, S. Wang, X. Ma, M. K. Ramanathan, and B. Ray, “Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 951–971, Jul. 2024.
- [12] S. Gu, N. Nashid, and A. Mesbah, “LLM Test Generation via Iterative Hybrid Program Analysis.” [Online]. Available: <http://arxiv.org/abs/2503.13580>
- [13] D. Liao, X. Yin, S. Pan, C. Ni, Z. Xing, and X. Sun. Navigating the Labyrinth: Path-Sensitive Unit Test Generation with Large Language Models. [Online]. Available: <http://arxiv.org/abs/2509.23812>
- [14] P. Godefroid, N. Klarlund, and K. Sen, “Dart: directed automated random testing,” in *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI ’05. New York, NY, USA: Association for Computing Machinery, 2005, pp. 213–223. [Online]. Available: <https://doi.org/10.1145/1065010.1065036>
- [15] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.

- [16] Y. Du, M. Tian, S. Ronanki, S. Rongali, S. Bodapati, A. Galstyan, A. Wells, R. Schwartz, E. A. Huerta, and H. Peng, "Context length alone hurts llm performance despite perfect retrieval," 2025. [Online]. Available: <https://arxiv.org/abs/2510.05381>
- [17] W. Wang, J. Min, and W. Zou, "Intelligence degradation in long-context llms: Critical threshold determination via natural length distribution analysis," 2026. [Online]. Available: <https://arxiv.org/abs/2601.15300>
- [18] M. Weiser, "Program slicing," *IEEE Trans. Softw. Eng.*, vol. 10, no. 4, pp. 352–357, Jul. 1984. [Online]. Available: <https://doi.org/10.1109/TSE.1984.5010248>
- [19] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Trans. Program. Lang. Syst.*, vol. 9, no. 3, pp. 319–349, Jul. 1987. [Online]. Available: <https://doi.org/10.1145/24039.24041>
- [20] C. Galindo, S. Perez, and J. Silva, "A program slicer for java (tool paper)," in *Software Engineering and Formal Methods*, B.-H. Schlingloff and M. Chai, Eds. Cham: Springer International Publishing, 2022, pp. 146–151.
- [21] R. Just, D. Jalali, and M. D. Ernst, "Defects4j: a database of existing faults to enable controlled testing studies for java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, ser. ISSTA 2014. New York, NY, USA: Association for Computing Machinery, 2014, pp. 437–440. [Online]. Available: <https://doi.org/10.1145/2610384.2628055>
- [22] T. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [23] Qwen Team, "Qwen3.5: Towards native multimodal agents," <https://qwen.ai/blog?id=qwen3.5>, Feb. 2026, official technical report / release article.
- [24] C. Watson, M. Tufano, K. Moran, G. Bavota, and D. Poshyvanyk, "On learning meaningful assert statements for unit test cases," in *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 1398–1409.
- [25] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, "Unit test case generation with transformers," *ArXiv*, vol. abs/2009.05617, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:235165921>
- [26] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, Jul. 2018. [Online]. Available: <https://doi.org/10.1145/3212695>
- [27] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 919–931.
- [28] J. Altmayer Pizzorno and E. D. Berger, "Coverup: Effective high coverage test generation for python," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2897–2919, 2025.
- [29] R. Dong, Z. Deng, X. Zhu, X. Du, H. Liu, S. Wang, S. Wen, and Y. Xiang, "Failmapper: Automated generation of unit tests guided by failure scenarios," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 2388–2400.
- [30] Y. Xie, A. Goyal, W. Zheng, M.-Y. Kan, T. P. Lillicrap, K. Kawaguchi, and M. Shieh, "Monte carlo tree search boosts reasoning via iterative preference learning," 2024. [Online]. Available: <https://arxiv.org/abs/2405.00451>
- [31] B. Chu, Y. Feng, K. Liu, H. Shi, Z. Nan, Z. Guo, and B. Xu, "Palm: Synergizing program analysis and llms to enhance rust unit test coverage," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 2720–2732.
- [32] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang, "Lsprag: Lsp-guided rag for language-agnostic real-time unit test generation," in *Proceedings of the 48th International Conference on Software Engineering*, ser. ICSE '26. New York, NY, USA: Association for Computing Machinery, 2026.
- [33] F. Tip, "A survey of program slicing techniques," *J. Program. Lang.*, vol. 3, no. 3, 1995. [Online]. Available: <http://compsocnet.dcs.kcl.ac.uk/JP/jp030301.abs.html>
- [34] A. Yadavally, Y. Li, S. Wang, and T. N. Nguyen, "A Learning-Based Approach to Static Program Slicing," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 83–109, Apr. 2024.
- [35] K. K. Shahandashti, M. M. Mohajer, A. B. Belle, S. Wang, and H. Hemmati, "Program slicing in the era of large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2409.12369>
- [36] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," in *Findings of the association for computational linguistics: EMNLP 2020*, 2020, pp. 1536–1547.